

CSE107 — Lab Makeup

Directory Polling & Language-Based Routing

Lab Date: 26/12/2025 — Due: 03/01/2026 11:59 pm

Student: Zeynep Özge TOPTAŞ
Student ID: 240104004006
Section: Section2
Instructor: Zeliha Erim

1. Objectives

This lab implemented a directory polling system that scans an "incoming" folder every 10 seconds, processes HTML files by extracting text, detects Turkish characters using UTF-8 analysis, and routes output to language-based directories. The implementation focused on handling filename collisions through counter-based naming, managing file operations safely (only deleting originals after successful processing), and robust error handling that preserves source files if any stage fails. Key technical practices included POSIX file system APIs, dynamic memory management, and character encoding concepts.

2. Environment

- **Operating System: Windows with WSL - Ubuntu**
- **IDE: Visual Studio Code**

3. Problem Solving

3.1 Handling large files during the HTML extraction was kind of hard but I used realloc to dynamically increase the buffer size as the file is read.

3.2 My code uses a while(1) loop with a sleep() function to check the "incoming" folder repeatedly. Polling is simpler to code but wastes CPU and has a delay. Checking the folder event-based would be efficient and instant, but much more complex to implement I suppose.

3.3 I use a for loop with sleep(1) to wait exactly 10 seconds. The time spent processing files (reading/writing) adds to the sleep time. If processing takes 2 seconds, the total loop becomes 12 seconds, causing "drift."

3.4 To prevent issues, I would move the file to a "working" folder before starting so other programs can't touch it. To detect if a file changed, I could use the stat function to check if the "last modified" timestamp changed while my program was running.

3.5 I skip all text between < and >. This works for basic files but fails if the HTML contains internal CSS, JavaScript, or angle brackets inside comments.

3.6 The program counts characters that are not English letters (A-Z). If it finds more than five, it classifies the file as Turkish.

3.7 ASCII only supports 128 English characters. UTF-8 is multi-byte and supports global characters. Turkish characters aren't in ASCII, so a simple 7-bit check misses them.

3.8 Byte-by-byte reading is insufficient because UTF-8 characters use multiple bytes. unsigned char is necessary because signed char treats values over 127 as negative, causing logic errors or infinite loops.

3.9 I use access() to see if a file exists and append a counter (like _2) if it does. This prevents overwriting existing data, ensuring every file is preserved.

3.10 If an error occurs, the program stops and keeps the original file in "incoming." This prevents data loss, as remove() is only called after the output is successfully saved.

4. Labwork Evidence — Screenshots

I developed a C program that periodically scans an "incoming" folder for HTML files using a polling loop. For each file found, the program extracts the plain text by removing HTML tags and checks for Turkish characters to determine the language. Based on this, it routes the processed text to either the "processed_english" or "processed_turkish" folder while handling naming conflicts. To ensure data safety, the original file is only deleted if the entire process completes without any errors.

```
ztoptas@MSI:/mnt/c/Users/msi/OneDrive/Desktop/CSE101/prelabs/make_up$ ./a.out
starting

--- scanning incoming/ directory ---
no .html files found in incoming/ directory right now
waiting 10 seconds before next scan...
1
2
3
4
5
6
7
8
9
10
```

```
--- scanning incoming/ directory ---
found file: english.html
successfully processed: english.html -> processed_english/english_processed.txt
found file: turkish.html
successfully processed: turkish.html -> processed_turkish/turkish_processed.txt
waiting 10 seconds before next scan...
1
2
3
4
5
6
7
8
9
10

--- scanning incoming/ directory ---
no .html files found in incoming/ directory right now
waiting 10 seconds before next scan...
```